Project Report – Database Analytics Design
Relational Database Design for Custodian Bank Data Analysis

Introduction

The purpose of this report is to propose an AI assisted database for a custodian bank and its Corporate Trust team supporting Collateralised Loan Obligations (CLOs). CLOs are securitised investment vehicles backed by floating rate loans with different levels of risk exposure (BlackRock, 2026). CLO administration involves large volumes of data relating to deals, obligors, assets, ratings and portfolio tests. Custodian banks offer trustee, collateral administration, loan administration and reporting services (BNY, 2026). The proposed solution aims to consolidate CLO data into a relational database, improving data quality, retrieval and analytical reporting.

AI Tools and Data

OpenAI's ChatGPT was selected as the main AI tool, supported by Python for data preparation and cleaning, SQLite for the prototype database and Power BI for visualisations. ChatGPT will be used to assist with schema design, generation and explanation of Python and SQL code and interpretation of validated outputs. Code generated by ChatGPT will be reviewed and tested by the development team before implementation.

As described in Sarkar and Roychowdhury (2019) data wrangling is the process of transforming raw data into data suitable for analysis through cleaning, restructuring and formatting. This is especially relevant because CLOs source files from a number of clients using various systems may include missing or inconsistent values, labels, and incompatible formats. In a real world scenario, confidential data would require

a company approved AI environment, encryption and audit logging to ensure data safety controls. As confidential client data cannot be used for this project, the prototype can use a synthetic dataset representing CLO deals, assets, trades and portfolio tests.

Logical Database Design

The proposed entities are:

CLODeal – DealID, DealName, ClosingDate, ManagerID

CollateralManager – ManagerID, ManagerName, ClientServiceLeadName

Obligor – ObligorID, ObligorName, FitchIndustry, S&PIndustry, Moody'sIndustry, Country

Asset – ObligorID, AssetName, AssetID, AssetType, ObligorLevelRating, AssetLevelRating, Spread, Coupon, MaturityDate

Trade – TradeID, DealID, AssetID, TradeType, TradeParAmount, TradePrice, TradeAmount, TradeDate, SettleDate, Currency

PortfolioTest – TestID, DealID, TestDescription, TestDate, TestThreshold, TestResultValue, TestResult

Proposed relational model

CollateralManager 1:M CLODeal

Obligor 1:M Asset

CLODeal 1:M Trade

Asset 1:M Trade

CLODeal 1:M PortfolioTest

The relational model separates clients, assets, transactions and tests to reduce duplication and improve referential integrity. DealID and AssetID act as foreign keys in Trade, while ObligorID connects each asset with its obligor.

Primary keys uniquely identify records, while foreign keys maintain valid relationships and ensure no references to non existent assets or deals.

Data Cleansing and Transformation

Firstly, Python will validate source files before loading them into the database. Duplicated TradeIDs, missing mandatory fields, invalid dates, and incorrect data types will be identified, along with unmatched DealIDs and AssetIDs. All dates, prices, par amounts and text will be converted into a consistent format.

Ratings and Industries will require further control. Industries like ‘Business Services’ and ‘Business Services General’ will be merged through a standardised documented mapping table. Moody’s, S&P and Fitch ratings will remain separately identifiable because differences may be legitimate, and ratings can change over time.

Variables will include net traded par, buy and sell volume, and test results headroom. For tests with a higher result being favourable, headroom can be calculated using the test result value minus the test threshold. This classifies tests as comfortable pass, approaching threshold or fail. Python and SQL will be used to calculate derived variables so that it is auditable.

Database Model Design

The proposed data pipeline is as follows:

Source Excel files -> Python -> SQLite -> SQL -> Power BI -> ChatGPT summary

Python and its Pandas library will be used to clean and standardise incoming files, apply mapping tables, and load validated data into SQLite. The database will be tested throughout development. Checks will include duplicated TradeIDs, missing references between tables, invalid dates and invalid numerical values. Total trade pars will be reconciled with source files to confirm data has not changed during transformation. Dashboard figures will be checked against the database before presenting results.

Power BI will be used to create a visual dashboard. Main KPIs at the top of the dashboard will show the total trade volume, number of trades and assets, and number of portfolio tests failing or approaching their limit. Charts will show trading activity by industry and country, ratings distribution and the most actively traded obligors. Filters will allow users to filter by individual CLO portfolios.

ChatGPT will then use the dashboard results to generate a written summary. It will highlight increased industry and country concentrations, and portfolio tests moving closer to the threshold. The actual calculations will be done by Python, SQL and Power BI, and ChatGPT will be used only to help explain these. This approach positions the AI assistant as a development and interpretation assistant rather than a system of autonomous decision making.

Critical Evaluation

ChatGPT can assist with database schema suggestions, Python and SQL code, code error explanations and provide explanations and data summaries. Additionally, the explanation in human like language would make analytical insights more accessible to users with less technical or CLO knowledge.

However, its output can include incorrect assumptions and fabricated information that contradicts CLO deal specific documents. Moreover, NIST (2024) identifies privacy and information security risks associated with generative AI models like ChatGPT.

Business rules should be defined rather than assumed by the AI system. Internal identifiers like AssetID, PortfolioID, ObligorID may differ between client and custodian bank systems, making controlled mapping and data lineage essential.

Moreover, the data processed is considered confidential, and appropriate security and encryption controls are needed. An institutionally approved and controlled AI environment for confidential data will be needed to implement this design. An example of this is the collaboration between OpenAI and custodian bank BNY, which allowed its employees to access a governed generative AI platform, Eliza (OpenAI, 2025).

Conclusion

The proposed AI assisted database for a custodian bank combines Python data wrangling, SQL database and data extraction, Power BI dashboard and use of ChatGPT. It considers realistic CLO data quality, cleaning, mapping and transformation challenges while demonstrating how AI can support and summarise key findings. Human intervention, code review, testing and strong data security controls are essential. Overall, the proposed design provides cleaner, structured data that is easier to interpret and ideal for analytical insights within the Corporate Trust department.

References:

BlackRock. (2026) *BlackRock – Demystifying Collateralized Loan Obligations (CLOs)*. Available at: https://www.blackrock.com/us/financial-professionals/insights/what-are-clos (Accessed 1 September 2026).

BNY. (2026) *Structured Debt: CLO & Loan Solutions*. Available at: https://www.bny.com/corporate/global/en/solutions/platforms/structured-debt-solutions/clo-loan-solutions.html (Accessed 1 September 2026).

National Institute of Standards and Technology. (2024) *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. Available at: https://doi.org/10.6028/NIST.AI.600-1.

OpenAI. (2025) *BNY Builds "AI for Everyone, Everywhere" with OpenAI*. Available at: https://openai.com/index/bny/ (Accessed 1 September 2026).

Sarkar, T. and Roychowdhury, S. (2019) *Data Wrangling with Python*. Birmingham: Packt Publishing. Available at: https://www.oreilly.com/library/view/data-wrangling-with/9781789800111/ (Accessed 31 August 2026).